

PowerShell File Integrity Hash Calculator

Trenton Carter – COMP 325, 10/16/2025

Overview

File integrity verification is an essential part of digital forensics and incident response. Cryptographic hashes allow investigators to detect tampering, verify evidence integrity, and compare files against threat intelligence. This lab uses a PowerShell script, `Get-FileHashReport.ps1`, to automate multi-algorithm hashing (MD5, SHA1, SHA256) across all files inside a target directory. The script collects file metadata, computes hash values, and exports results to a timestamped CSV report.

Execution

1. Script Initialization

The script defines `Get-FileHashReport`, which accepts a directory path and list of hashing algorithms. When executed, it prints the selected path and algorithms and prepares for recursive scanning.

```
# File Hash Calculator for Forensics
function Get-FileHashReport {
    param(
        [string]$Path = ".",
        [string[]]$Algorithms = @("MD5", "SHA1", "SHA256")
    )

    Write-Host "=== File Hash Report Generator ==="
    Write-Host "Path: $Path"
    Write-Host "Algorithms: $($Algorithms -join ', ')"
    Write-Host ""

    # Get all files in the specified path
    $Files = Get-ChildItem -Path $Path -File -Recurse
    $HashResults = @()

    foreach ($File in $Files) {
        Write-Host "Processing: $($File.Name)"
        $FileHashes = @{
            'FileName'      = $File.Name
            'FullPath'      = $File.FullName
            'Size (bytes)'  = $File.Length
            'LastModified'  = $File.LastWriteTime
        }

        # Calculate hashes for each algorithm
        foreach ($Algorithm in $Algorithms) {
            try {
                if ($PSVersionTable.PSVersion.Major -ge 5) {
                    # Use Get-FileHash if available (PowerShell 5+)
                    $Hash = (Get-FileHash -Path $File.FullName -Algorithm $Algorithm).Hash
                }
                else {
                    # Fallback for older PowerShell versions
                    $Hash = "Not Available (PowerShell < 5.0)"
                }
                $FileHashes[$Algorithm] = $Hash
            }
            catch {
                $FileHashes[$Algorithm] = "Error: $($_.Exception.Message)"
            }
        }

        $HashResults += [PSCustomObject]$FileHashes
    }

    # Display results
    $HashResults | Format-Table -AutoSize

    # Save to CSV
    $ReportFile = "file_hashes_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').csv"
    $HashResults | Export-Csv -Path $ReportFile -NoTypeInformation
    Write-Host "`nHash report saved to: $ReportFile"

    return $HashResults
}

# Generate hash report for current directory
Get-FileHashReport -Path "."
```

2. File Enumeration

All files within the directory (and subdirectories) are collected using PowerShell's recursive file search. For each file, metadata is recorded including:

- File name
- Full path
- Size (bytes)
- Last modified date

```
(kali㉿kali)-[/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis]
PS> ./Get-FileHashReport.ps1
=== File Hash Report Generator ===
Path: .
Algorithms: MD5, SHA1, SHA256

Processing: AnalyzeSecurityFile.ps1
Processing: config.txt
Processing: EventLogAnalysis.ps1
Processing: Get-FileHashReport.ps1
Processing: incident.md
Processing: IncidentResponseCollector.ps1
Processing: Invoke-PortScan.ps1
Processing: LogSecAnalysis.ps1
Processing: NetworkScanSim.ps1
Processing: NetworkSecurityAssessment.ps1
Processing: PasswordComplexityCheck.ps1
Processing: port_scan_results.csv
Processing: PowerShellVariables.ps1
Processing: security.log
Processing: SecurityAssessment.ps1
Processing: SecurityLogReview.ps1
Processing: windows_events.csv
Processing: incident_summary.txt
Processing: processes.csv
Processing: system_info.json
```

3. Hash Calculation

For each file, the script loops through MD5, SHA1, and SHA256:

- If running PowerShell 5+, it uses Get-FileHash
- If unsupported, it records “Not Available”
- Any errors (e.g., access denied) are captured rather than stopping execution

This creates a complete record of file metadata plus its hash values.

SHA1	LastModified	MD5	FullPath	SHA256
386C045E24938517C531B24E9C894B9ABA9A3AC	10/16/2025 10:41:33 PM	75E1D1D9F3CF7638589C9D00AAFF8AE	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/AnalyzeSecurityFile.ps1	E110A1F5...
3399E0DAED47F2467046F85C90AF2708BA5983	10/16/2025 10:27:07 PM	84FD7610F6E0E0911A9F8C73BA184CDE	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/config.txt	830983D3...
1027F8E07E3CD720B0A27C472AB2C418E7924A7	10/16/2025 11:11:06 PM	777FF0A0639329AF2F40C9F808C991A7	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/EventLogAnalysis.ps1	4736876A...
24217D611898536A8A5DA43519744A0A23D18058	10/17/2025 12:30:58 AM	BC7C923E4A5CA1256B2E04F77AE071FF	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/Get-FileHashReport.ps1	7EB27227...
0A4F2C01F90329E48E997F96F07A3B788A4F0	10/16/2025 10:27:07 PM	DA03292A5A86D3F8EF77FB6E97733A	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/incident.md	D1FCF589...
CBFE58B6182096894C38B7F1E4C0207A53417CF	10/17/2025 12:24:25 AM	4AFA1AC4390511448F678451E92514AD	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/IncidentResponseCollector.ps1	E02AF2B3...
1F00818D1482335141901F2F3B12A967872012	10/17/2025 12:28:21 AM	071C8283853760D42A64E4047E0E85A3	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/Invoke-PortScan.ps1	E09ED923...
0107EE5F934E87FCFE77A5B0994FF3D10808604	10/16/2025 10:32:20 PM	B55F8BF9D2AA7CD5227388207A17300F	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/LogSecAnalysis.ps1	98F771A9...
025A66631CC84216812D127C7CA367ED0C61A48	10/16/2025 10:33:50 PM	26E922B4298BF1A27B9533007593CC49	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/NetworkScanSim.ps1	8856379A...
8B77A43E877F4B763679C008C5612AD8521A9E7F	10/17/2025 12:21:24 AM	178141CF0508E830FA7783F38365774	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/NetworkSecurityAssessment.ps1	D22982F5...
1B8BDE5F6CF28C489A01D985061272058CFFA4C2	10/16/2025 11:59:28 PM	E07EE4E7897E2E452172D117D13A417	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/PasswordComplexityCheck.ps1	66F9208C...
0910A070DE51D9585132A49460A4133C0A0026A	10/17/2025 12:28:38 AM	ED0520CD7B8BFA5C38CA00841DE32B12	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/port_scan_results.csv	D4F4204B...
0072CCE19018C39C46970E23F947221600ED7A	10/16/2025 10:33:50 PM	82111D2A3845D0155265902B8614C51	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/PowerShellVariables.ps1	85F7A216...
7CE0D51F02883C658F0054CD07AD075EC5F879	10/16/2025 10:32:22 PM	4E1AD7158840BFDE3EB12A5615F08D3	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/security.log	454DB5CF...
3B3498DA27094FE08C09E96C02D7F8C19E145C5	10/16/2025 10:55:54 PM	56295399F15C3BE64302E0E00EF01F60	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/SecurityAssessment.ps1	9E41D29B...
00538107170794FA6838D410EAB4587A0D20C2	10/16/2025 10:36:59 PM	4AD98BF8FD2081EF1E84E6A4C1CD7C899	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/SecurityLogReview.ps1	76582E09...
981C66AFA4C125E72A5D0E65C0F6862C06C47C	10/16/2025 11:11:23 PM	B781B88F8A4A5701678E8B15CF0E2007	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/windows_events.csv	A8F0AF6E...
DB8EC9F508FB46E7F781F7D3BC10929D22E69E7	10/17/2025 12:24:50 AM	6AKBD57E49413085B7CAEA6249C2129	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/incident_data/incident_summary.txt	CB8C02EF...
8AC52CFB86A182E26F37083AFA10382A12F1F99E	10/17/2025 12:24:50 AM	34F53232B787A2F0B862F68E3A6F2460	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/incident_data/processes.csv	483AB1E0...
16EC557CBED0859E506E21A5DBA6CF21763D2A	10/17/2025 12:24:50 AM	B50A800F590357395E31F7011082C20	/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/incident_data/system_info.json	5620097E...

4. CSV Report Generation

After processing all files, the script generates a timestamped CSV file:

The CSV includes metadata and all computed hashes for use in forensic tools, spreadsheets, or SIEM systems.

```

LastModified : 10/16/2025 10:41:31 PM
MD5          : 73EB1B1DF9C5F7638569C9DDDAFF8AE
FullPath     : /home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/AnalyzeSecurityFile.ps1
SHA256       : E110A1F522E4412FD7D887476ACA4E4B6DB5C0441C31488D4B7B03BB50722C75
FileName     : AnalyzeSecurityFile.ps1
Size (bytes) : 898

SHA1         : C399EDB0AEDA7F82A67046FB5C90AF2708BA59B3
LastModified : 10/16/2025 10:27:07 PM
MD5          : BAFD7610F60ED6911A9FBC73BA184CDE
FullPath     : /home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/config.txt
SHA256       : 830993D3855CCB70E133B55D70F8EA91ED41AFBB6A90F95C9E3F74CFE67B4C16
FileName     : config.txt
Size (bytes) : 19

SHA1         : 1027FBE07E3CD720BD0A27C472A8C2418E7924A7
LastModified : 10/16/2025 11:11:06 PM
MD5          : 777FF0A0639329AF2F40C9F80BC991A7
FullPath     : /home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/EventLogAnalysis.ps1
SHA256       : 4736876A60068132904FB86114F78D9C86AB0BE6FA734B46D021BC38D779E238
FileName     : EventLogAnalysis.ps1
Size (bytes) : 1350

SHA1         : 24217D611898536AA85DA4351974A4DA23D1B05B
LastModified : 10/17/2025 12:30:58 AM
MD5          : BC7C923E4A5CA1256B2ED4F77AED71FF
FullPath     : /home/kali/cybersec-labs/powershell-lab/SecurityAnalysis/Get-FileHashReport.ps1
SHA256       : 7E8272272DB376ED1FD9B5C1B9E250E8F02C424EA7F8C825AAE1FB6F6A53E0DA
FileName     : Get-FileHashReport.ps1
Size (bytes) : 1872

```

Findings & Results

The script successfully produced a full hash report for every file in the directory. The results provide investigators with reliable evidence for integrity checks, tamper detection, and malware comparison. The timestamped CSV makes it easy to maintain versioned snapshots during an investigation.

Skills Demonstrated

- PowerShell automation
- Recursive directory enumeration
- MD5, SHA1, and SHA256 hashing
- Error-tolerant forensic scripting
- CSV export and evidence documentation

Conclusion

This lab demonstrated how PowerShell can be used to automate file integrity analysis in a forensic workflow. By generating hash values for large sets of files and exporting them to a structured CSV report, the Get-FileHashReport.ps1 script provides a fast, repeatable method for verifying evidence integrity and detecting unauthorized file modifications.